

ПРЕРЕКВИЗИТЫ

Вспоминаем инструменты

git, терминал и как устроен запрос к серверу. Это фундамент, на котором стоит весь курс, но который на лекциях не читается. Прочитать — 15 минут, плюс диагностика Python отдельным ноутбуком.

Git за одну страницу

Весь код курса живёт в git-репозитории, а не в ячейках ноутбука. Причина простая: домашки сдаются ссылкой на репозиторий, преподаватель смотрит историю, а на четвёртой неделе мульти-агентная система в ячейках превращается в нечитаемое полотно.

Шесть команд, которых хватит на весь курс

```
git clone <url>      // скопировать репозиторий к себе
git status           // что изменилось
git add .            // отметить изменения к сохранению
git commit -m "что сделал" // сохранить точку в истории
git push             // отправить на GitHub
git pull             // забрать чужие изменения
```

Рабочий цикл: поменяли код → `git add .` → `git commit -m "..."` → `git push`. И так после каждого осмысленного куска работы, а не раз в неделю.

Одна строчка, которая спасёт вас от беды

Создайте в репозитории файл `.gitignore` и впишите туда `.env` **до первого коммита**. Иначе ваш ключ уедет на GitHub, где его за минуты найдут боты. Туда же обычно кладут `.venv/`, `__pycache__/`, `*.pyc`.

Если git ещё не стоял

Поставьте с git-scm.com, один раз представьтесь:

```
git config --global user.name "Ваше Имя"
git config --global user.email "you@example.com"
```

Аккаунт на github.com тоже понадобится — заведите заранее.

Терминал и окружение

Курс пишется в файлах `.py` и запускается из терминала, а не только в ячейках. Если вы жили в Jupyter — вот необходимый минимум.

- **Где я нахожусь и что вокруг:** `pwd` (текущая папка), `ls` (список файлов; в Windows PowerShell тоже работает), `cd имя_папки` (зайти внутрь), `cd ..` (на уровень выше).
- **Виртуальное окружение** — изолированный набор пакетов под проект:

```
python -m venv .venv
.venv\Scripts\activate // Windows
source .venv/bin/activate // macOS / Linux
```

После активации в начале строки появляется `(.venv)`. Пакеты ставятся сюда, а не в систему.

- **Установка пакетов:** `pip install requests pydantic python-dotenv`
- **Запуск скрипта:** `python agent.py`. Это не то же самое, что «выполнить ячейку»: скрипт обрабатывает целиком от начала до конца.

Скрипт против ноутбука

Ноутбук хорош, чтобы поэкспериментировать и посмотреть на данные. Но агент — это программа, которая работает без вас, поэтому его код живёт в `.py`-файлах. На курсе держите ядро агента в файлах, а ноутбук используйте для запусков и разбора результатов.

HTTP и JSON: как код разговаривает с моделью

Модель живёт на чужом сервере. Ваш код общается с ней по протоколу HTTP: отправляет **запрос** и получает **ответ**. Весь курс стоит на одном типе запроса, поэтому разберём его устройство.

Запрос

- **Метод** `POST` — «отправляю данные и жду ответ» (в отличие от `GET` — «просто дай почитать»).
- **URL** — адрес, куда стучимся: `https://openrouter.ai/api/v1/chat/completions`
- **Заголовки** (headers) — служебная информация, в том числе ключ: `Authorization: Bearer <ключ>`
- **Тело** (body) — сами данные в формате JSON: какая модель, какие сообщения, сколько токенов.

JSON — это словарь Python

JSON выглядит как питоновский словарь и почти им и является: пары «ключ: значение», списки в квадратных скобках, вложенность. Библиотека `requests` сама превращает ваш словарь в JSON на отправку и обратно на приём.

```
{"model": "openai/gpt-4o-mini",
 "messages": [{"role": "user", "content": "Привет"}],
 "max_tokens": 50}
```

Ответ и статус-коды

Сервер возвращает JSON с ответом и число — статус-код, по которому видно, что случилось:

- **200** — всё хорошо, ответ внутри.
- **401** — не приняли ключ (неверный или не передан).
- **402** — кончились кредиты или бесплатный лимит.
- **429** — слишком часто, подождите и повторите.
- **500–503** — упал сервер провайдера, повторите с паузой.

Как из ответа достать текст и токены — отдельным ноутбуком `razbor_otveta.ipynb`. Он показывает ту самую строчку `response['choices'][0]['message']['content']` по шагам.

Что сделать сегодня

1. Прочитать этот лист (15 минут).
2. Пройти `python_diagnostika.ipynb` — десять микрозадач с самопроверкой. Если тяжело, внутри ссылки, что добрать.
3. Пробежать `razbor_otveta.ipynb` — как разобрать ответ API. Сети не требует.

4. Если ещё нет — завести аккаунт GitHub и поставить git.

Дальше: вспоминаем, как вообще устроены языковые модели.